

Test 3 Practice — LC-3 Programming NAME:

Multiple Choice Problems (Each one, 2 points)

1. In LC-3, the **JSR** instruction stores the return address in which register?
 - A) R0
 - B) R5
 - C) R6
 - D) R7
2. Which of the following is the correct sequence for a PUSH on a downward-growing stack?
 - A) Store the value, then decrement R6
 - B) Decrement R6, then store the value
 - C) Increment R6, then store the value
 - D) Store the value, then increment R6
3. What does **LEA R0, LABEL** load into R0?
 - A) The value stored at LABEL
 - B) The value stored at the address contained in LABEL
 - C) The address of LABEL
 - D) The value x0000
4. The **PUTS** trap (x22) requires R0 to contain:
 - A) The ASCII value of a single character
 - B) The address of a null-terminated string
 - C) The number of characters to print
 - D) The ASCII value of the first character
5. **BRnp** branches when the last result was:
 - A) Negative and positive simultaneously
 - B) Not zero (either negative or positive)
 - C) Negative only
 - D) Zero only
6. When a trap handler uses R1 and R2 as scratch registers, it must:
 - A) Clear R1 and R2 to zero before using them
 - B) Save R1 and R2 on the stack and restore them before RTI
 - C) Do nothing — the hardware saves all registers on TRAP entry
 - D) Only save R7
7. What does **LDI R0, PTR** do when the memory at PTR contains x4000?
 - A) Loads the address of PTR into R0
 - B) Loads x4000 into R0
 - C) Loads the value stored at address x4000 into R0
 - D) Loads x0000 into R0
8. User-defined trap handler code should be placed at:
 - A) x0000–x00FF (trap vector table)
 - B) x3000 (user program space)
 - C) x0200 or higher in OS/supervisor space (e.g., x0300)
 - D) xFE00–xFFFF (device registers)
9. **RET** in LC-3 is equivalent to:
 - A) **JMP R0**
 - B) **JMP R6**
 - C) **JMP R7**
 - D) **RTI**
10. Which trap reads a single character from the keyboard without echoing it, returning the ASCII value in R0?
 - A) TRAP x21 (OUT)
 - B) TRAP x22 (PUTS)
 - C) TRAP x20 (GETC)
 - D) TRAP x23 (IN)

Assembly Language Problems (Each one, 15 points)

The following problems will be awarded points in the following manner:

Total 60 points — Four LC-3 Assembly Language problems, 15 points each.

- 0–5 points — restate the problem in comments at the top
 - 0–5 points — correct syntax, neatness and well presented
 - 0–5 points — problem was solved (algorithm was correct)
 - **0–15 points total per problem**
-

Problem 1

Write an LC-3 Assembly Language program that prints each character of the string `"LC3"` to the console one at a time using the `OUT` trap (`TRAP x21`). Use a loop that advances through the characters until the null terminator is found. **Do not use `PUTS`** .

The string is defined as:

```
MSG      .STRINGZ "LC3"
```

Problem 2

Write an LC-3 Assembly Language program that counts the number of non-zero elements in the following array of six words and stores the count in a variable called **NONZERO**.

```
VALUES      .FILL #-3  
            .FILL #7  
            .FILL #0  
            .FILL #5  
            .FILL #-1  
            .FILL #4
```

Problem 3

Write an LC-3 subroutine called `ABS_VAL` that accepts a signed integer in R0 and returns its absolute value in R0. Save and restore any registers the subroutine modifies (except R0).

Write a brief test program at `.ORIG x3000` that:

1. Initializes the user stack pointer (R6 = x4000)
2. Loads the value `#-5` into R0
3. Calls `ABS_VAL` using `JSR`
4. Stores the result in a variable called `RESULT`

Problem 4

Write a complete user-defined trap at vector `x0027` that converts the value in R0 to an ASCII number. Assume the value in R0 will be between #0-9.

Provide **all three required sections**:

1. **Trap vector table entry** — a `.ORIG x0027` block that stores the handler address
2. **Handler code** — placed at `.ORIG x0300`
3. **Test program** — at `.ORIG x3000`, loads `#5` into R0, calls `TRAP x27`, and stores the result in a variable called `ASCII`

Debugging Problems (Each one, 5 points)

1. When I attempt to run this program, the simulator immediately reports:

```
--- Access Violation ---
```

The error occurs as soon as the subroutine starts executing. **Please explain what the problem is.**

```
.ORIG x3000
LD  R0, INPUT
JSR DOUBLE
ST  R0, RESULT
HALT

INPUT    .FILL #6
RESULT   .BLKW 1

DOUBLE   ADD  R6, R6, #-1    ; save R1 on stack
          STR  R1, R6, #0
          ADD  R1, R0, R0
          ADD  R0, R1, #0
          LDR  R1, R6, #0    ; restore R1
          ADD  R6, R6, #1
          RET
          .END
```

2. The subroutine below saves R0 and R1 before doing computation, then restores them before returning. After the call, the user program finds that R0 has R1's original value and R1 has R0's original value — they are swapped. **What is wrong?**

```
HANDLER   ADD  R6, R6, #-1
          STR  R0, R6, #0    ; push R0
          ADD  R6, R6, #-1
          STR  R1, R6, #0    ; push R1

          ; ... computation using R0 and R1 ...

          LDR  R0, R6, #0    ; restore
          ADD  R6, R6, #1
          LDR  R1, R6, #0    ; restore
          ADD  R6, R6, #1
          RTI
```

3. The subroutine **OUTER** is supposed to return to whoever called it. Instead the program jumps to the wrong location after **OUTER** returns. **What is the problem?**

; OUTER: computes $R0 = (R0 * 2) + R2$, using **INNER** as a helper

```

OUTER      ST  R2, SAVE_R2
           ADD R2, R0, R0      ; R2 = R0 * 2
           JSR INNER          ; call helper subroutine
           LD  R2, SAVE_R2
           RET                 ; return to original caller

INNER      ADD R0, R0, R2      ; R0 = R0 + R2
           RET

SAVE_R2    .BLKW 1

```

4. This program should sum exactly 5 elements from the array, but it reads one extra element and produces the wrong result. **What is the bug?**

```

           LD  R1, COUNT      ; R1 = 5
           LEA R2, VALS       ; R2 = base address
           AND R0, R0, #0     ; R0 = sum

LOOP       LDR R3, R2, #0     ; load element
           ADD R0, R0, R3     ; accumulate
           ADD R2, R2, #1     ; advance pointer
           ADD R1, R1, #-1    ; decrement count
           BRzp LOOP         ; loop while count >= 0
           ST  R0, SUM

           HALT

COUNT     .FILL #5
VALS       .FILL #2
           .FILL #4
           .FILL #6
           .FILL #8
           .FILL #10

SUM        .BLKW 1
           .END

```